# for msrit qml code on 8826

sarmanarasimha

8th August 2026

def greet(name):

## Encoding a vector into quantum state

starting from a classical vector v=$[v_0, v_1, v_2, v_3]$ Normalization $\implies \langle\psi|\,\psi\rangle = 1 \implies$

$$v^{/} = \frac{v}{\sqrt{v_0^2 + v_1^2 + v_2^2 + v_3^2}}$$

when normalized vector is encoded into a 2-bit quantum state

$$|\psi\rangle = v_0^{/}\,|00\rangle + v_1^{/}\,|01\rangle + v_2^{/}\,|10\rangle + v_3^{/}\,|11\rangle$$

```
    #qiskit simulation example
    from qiskit import Aer,execute
    #simulate the defined circuit
    simulator = Aer.get_backend('aer_simulator')
    result = execute(qc,simulator).result()
    #get and display the result
    counts=result.get_counts()
    print("Simulation Result:",counts)
    #this one for upcomin section 'qc' example

# qiskit version 1.0
from qiskit import QuantumCircuit
from qiskit.visualization import circuit_drawer
import numpy as np
#classical encoding
v=np.array([1,2,3,4])
#Normalization of
v=np.linalg.norm(v)
v_normalized=v/norm
# quantum circuit creation
qc=QuantumCircuit(2)
#encode into quantum state
qc.initialize(v_normalized,[0,1])
```

```
print("QuantumCircuit for vector encoding:")
print(qc)
```

## Cirq example

```
import cirq
#create 2 qubits
q0,q1=cirq.LineQubit.range(2)
#circuit definition
Circuit=cirq.Circuit(
cirq.H(q0),
cirq.CNOT(q0,q1),
cirq.measure(q0,q1)
)
#sumulation of circuit
simulator=cirq.Simulator()
result=simulator.run(circuit)
print("CirqSimulation Result:")
print(result)
```

## pennylane(Hybrid quantum-classical workflows)

- supports variational circuits
- interfaces with –TensorFlow,Pytorch

```
import pennylane as qml

#to define 2 qubit device
dev=qml.device("default.qubit",wires=2)
# to define quantum function
@qml.qnode(dev)
 def circuit(params):
   qml.RY(params[0],wires[0])
   qml.RY(params[1],wires=1)
   qml.CNOT(wires=[0,1])
   return qml.probs(wires=[0,1])

#Execxute the circuit with sample parameters
params=[0.5,1.0]
print("pennylane Simulation Result:")
print(circuit(params))
```

## Feature mapping

- transforms classical features into quantum states
- so, quantum algorithms can leverage high-dimensional spaces for better learning and classification
- quantum kernels use feature maps to transform classical data into quantum states
- $\phi(x) = \exp\left((i\sum_j x_j Z_j)\right)|0\rangle$
- $Z_j$–pauli z-gate appplied to the j-th qubit
- $x_j$—–j-th feature of data point
- `Benefits of feature mapping are`:
- Efficient representation of non-linear relationships
- Enables quantum-kernel based methods,like QSVM's

## Feature mapping example with qiskit 1.0

from qiskit import QuantumCircuits from qiskit.visualization import circuit_drawer import numpy as np

```
#Define a feature mapping function
def feature_map(data_point):
  n_qubits=len(data point)
  qc=QuantumCircuit(n_qubits)
  for i,feature in enumerate(data_point):
   qc.rz(feature,i) # Z-roatation in proportion to feature
value
    return qc
# data point (example)
data_point=np.array({0.5,1.0,1.5})
# feature map generation
qc=feature_map(data_point)
print("Quantum Circuit for Feature mapping":)
peint(qc)
Draw the circuit
display(circuit_drawer(qc,output='mpl',style="iqp")
```

## Encoding complex data structures

- QML applications also involve—High -dimensional / structured data
- Examples–categorical variables or time-series data
- ⇓
- in such case `Hybrid encoding techniques`
- Hybrid encoding combines both amplitude encoding and angle encoding to manage large data sets efficiently.
- `Amplitude encoding`—-numerical values
- encoded into amplitudes $x_i$ of a quantum state i.e
- $|\psi\rangle = \frac{1}{\sqrt{N}} \sum_{i=0}^{n-1} x_i \, |i\rangle$
- Angle encoding for categorical features
- `Angle Encoding`—- encoded into angles of qubit rotations
- Example—calssical value 'x' can be encoded as a rotation angle $\theta$ on the Bloch sphere:
- $R_y(\theta) = \cos\frac{\theta}{2} + \sin\frac{\theta}{2} \, |1\rangle$
- categorical features are mapped into discrete quantum states
- category A $\mapsto |00\rangle \, , B \mapsto |01\rangle \, , C \mapsto |10\rangle$

## Hybrid Encoding example

```
from qiskit import QuantumCircuit
from qiskit.visualization import circuit_drawer
import numpy as np
#Numerical features(Amp)
numerical_data=np.array([3,2,5,7])
norm = np.linalg.norm(numerical_data)
normalized_numerical=numerical_data/norm
#categorical features
categorical_data={"A":[0,0],"B":[0,1],"C":1,0[]}
category_state=categorical_date["B"]
#Creating Quantum circuit
qc=QuantumCircuit(3)#2 for numerical
qc.initialize(normalized_numerical,[0,1])# Amplitude encoding
qc.initialize(category_state,2)# basis encoding
print("Quantum Circuit for Hybrid Encoding:")
print(qc)
```

```
#Draw circuit
display (circuit_drawer(qc,output='mpl',style="iqp"))
```

## preprocessed quantum datasets are used for

- development of quantum codes
- QML models—in QSVMs, QNNs, VQC for classification and regression tasks
- `optimization problems`—- normalized data and feature maps are widely used in QAOA
- `Quantum feature Engineering`— hybrid encoding, kernel methods, used for extracting meaningful patterns from quantum data sets

## Quantum Feature mapping

- Quantum feature mapping encodes classical data into high-dimensional quantum states
- Advantage—enables efficient exploration of complex relationships
- Mathematical representation:
- $|\psi(x)\rangle = U_\phi(x)\,|0\rangle$ where,
- x(classical data point)$\mapsto$ $|\psi(x)\rangle$ quantum state using a Feature map ($|\phi(x)\rangle$)
- $U_\phi(x)$—unitary operation encoding the data into quantum state
- `kernel Trick in quantum Feature mapping`
- quantum kernel computes inner products in quantum feature space such that non-linear classification can be done.
- $\mathbf{K}(x, x') = |\langle\psi(x)|\,\psi(x')\rangle|^2$

## Quantum Feature mapping qiskit code Example

```
#quantum Feature map with Visualization
import numpy as np
import matplotlib.pyplot as plt
from qiskit import Aer,QuantumCircuit
from qiskit.circuit.library import ZZFeaturemap
from qiskit_machine_learning.kernels import QuantumKernel
from qiskit.visualization import circuit_drawer
```

```
#visualizing quantum circuit
def plot_quantum_circuit(qc):
  """ Function to plot the quantum cicuit"""
  print("\nQuantum Feature Map Circuit:")
  print(qc.decompose()) #print circuit decomposition
  circuit_drawe(qc,outout='mpl',style=(){'backgroundcolor':white})

  # Define a quantum feature map
  feature_map=ZZFeatureMap(feature_dimension=2,reps=2)

  #creation of quantum kernel
  quantum_kernel=QuantumKernel(feature_map=feature_map,
  quantum_instance=Aer.get_backend("aer_simulator'))

  #Executing visualization
  plot_quantum_circuit(feature_map)
```